3661. 可以被机器人摧毁的最大墙壁数目

难度: **Hard**

一条无限长的直线上分布着一些机器人和墙壁。给你整数数组 `robots` ， `distance` 和 `walls`：

- `robots[i]` 是第 `i` 个机器人的位置。
- `distance[i]` 是第 `i` 个机器人的子弹可以行进的 **最大** 距离。
- `walls[j]` 是第 `j` 堵墙的位置。

每个机器人有 **一颗** 子弹，可以向左或向右发射，最远距离为 `distance[i]` 米。

子弹会摧毁其射程内路径上的每一堵墙。机器人是固定的障碍物：如果子弹在到达墙壁前击中另一个机器人，它会 **立即** 在该机器人处停止，无法继续前进。

返回机器人可以摧毁墙壁的 **最大** 数量。

注意：

- 墙壁和机器人可能在同一位置；该位置的墙壁可以被该位置的机器人摧毁。
- 机器人不会被子弹摧毁。

示例 1:

输入: robots = [4], distance = [3], walls = [1,10]

输出: 1

解释:

- `robots[0] = 4` 向 **左** 发射， `distance[0] = 3` ，覆盖范围 `[1, 4]` ，摧毁了 `walls[0] = 1` 。
- 因此，答案是 1。

示例 2:

输入: robots = [10,2], distance = [5,1], walls = [5,2,7]

输出: 3

解释:

- `robots[0] = 10` 向 **左** 发射， `distance[0] = 5` ，覆盖范围 `[5, 10]` ，摧毁了 `walls[0] = 5` 和 `walls[2] = 7` 。
- `robots[1] = 2` 向 **左** 发射， `distance[1] = 1` ，覆盖范围 `[1, 2]` ，摧毁了 `walls[1] = 2` 。
- 因此，答案是 3。

示例 3:

输入: robots = [1,2], distance = [100,1], walls = [10]

输出: 0

解释:

在这个例子中，只有 `robots[0]` 能够到达墙壁，但它向 **右** 的射击被 `robots[1]` 挡住了，因此答案是 0。

提示:

- `1 <= robots.length == distance.length <=` 10^5
- `1 <= walls.length <=` 10^5
- `1 <= robots[i], walls[j] <=` 10^9
- `1 <= distance[i] <=` 10^5
- `robots` 中的所有值都是 **互不相同** 的
- `walls` 中的所有值都是 **互不相同** 的